

OneShetland — Payments & Payouts Test Plan (pre go-live)

Work through these in Stripe TEST mode first. Each story has: who/where, steps, what should happen, where to verify, and a box for your feedback. Once everything here passes, follow §M Go-live switch to move to live Stripe.

Legend: + Pass · + Fail ·

Verify in two places every time: (a) Stripe Dashboard (test mode) — Payments, Connect !' connected accounts, Balance; (b) Supabase — the DB row/table named in the story.

A. Pre-flight setup (do once before testing)

A1. Confirm TEST keys are set in Supabase Edge Function secrets: STRIPESECRETKEY (sktest...), STRIPEWEBHOOKSECRET, STRIPECONNECTWEBHOOKSECRET, and web NEXTPUBLICSTRIPEPUBLISHABLEKEY (pktest...).

A2. Confirm test Price IDs in adminconfig (or env): STRIPEPRICELOCALPRO, STRIPEPRICELOCALPREMIUM, STRIPEPRICEALERTADDON. (If missing, subscriptions fail with a cryptic error — known gap.)

A3. Stripe webhook endpoint points at {SUPABASEURL}/functions/v1/stripe-webhook and is subscribed to: paymentintent.succeeded, paymentintent.paymentfailed, account.updated, customer.subscription., invoice.paymentsucceeded, transfer.created.

A4. Have 3 test logins ready: a Customer, a Business owner, a Hub admin, plus a Driver (Fetch). You'll onboard their payouts in §I.

Stripe test cards: success 4242 4242 4242 4242 · decline 4000 0000 0000 0002 · 3-D Secure/auth required 4000 0025 0000 3155. Any future expiry, any CVC/postcode.

B. Card on file (foundation for everything)

B1 — Save a card · Customer · app payment-setup.tsx (Local !' Wallet/settings !' add card)

1. Add 4242.... 2. Save.

- ' Card saved; profiles.stripecustomerid set; card shows as saved on return.

Pass	Fail	Notes:
------	------	--------

B2 — Declined card · Customer · same screen, card 4000 0000 0000 0002

- ' Clear decline message; no card saved; no crash.

Pass	Fail	Notes:
------	------	--------

C. Wallet (top-up = money IN, pay = payout to business)

C1 — Top-up, new card (PaymentSheet) · Customer · local-wallet.tsx !' Top up £10 (usesavedcard off)

- ' PaymentSheet appears !' pay !' balance +£10. No platform fee (your own money in). DB: localwallettransactions type=topup; balance via walletcredit. Stripe: PaymentIntent £10, metadata type=localwallettopup.

Pass	Fail	Notes:
------	------	--------

C2 — Top-up, saved card (off-session, no UI) · Customer · Top up £20

- ' Charges saved card with no card form; balance +£20.

Pass

Fail

Notes:

C3 — Pay in store with wallet !' BUSINESS PAYOUT · Customer · local-pay.tsx enter business code + amount (e.g. £10)

- ' Wallet " £10. Business receives a Stripe transfer to its connected account = £10 " platform commission (2% + 25p default) " business's own cashback%. DB: localwallettransactions debit; business credited. Stripe: Connect !' business account shows the transfer; platform keeps the 2%+25p.
- & p This is the main customer!'business payout — check the maths carefully.

Pass

Fail

Notes:

C4 — Pay with insufficient balance · Customer · pay more than balance

- ' Rejected cleanly; balance unchanged.

Pass

Fail

Notes:

C5 — Pay a business that hasn't finished payout setup · (use a business with payoutenabled=false)

- ' Blocked with "has not finished setting up payouts yet"; wallet not debited.

Pass

Fail

Notes:

D. Event tickets (the 95p / commission flow !' organiser payout)

D1 — Buy tickets (saved card) · Customer · event-ticket-checkout.tsx for an event with a paying organiser

1. Pick 2 tickets. 2. Enter attendee details. 3. Pay.
- ' Charged total; tickets become valid; capacity decremented (no oversell). DB: eventticketorders status=paid, eventtickets status=valid, event tickets sold +2. Stripe: destination charge to organiser's Connect account; platform keeps commission. Push notification received.

Pass

Fail

Notes:

D2 — Oversell guard · two near-simultaneous buys of the last ticket(s)

- ' Only one succeeds; the other gets a clean "sold out". No negative capacity.

Pass

Fail

Notes:

D3 — Buy tickets, card declined · Customer · use 4000...0002

- ' No tickets issued; capacity released (pending order not left blocking stock).

Pass

Fail

Notes:

E. Hubs — membership, donation, Gift Aid (!' hub payout)

E1 — Join paid membership · Customer · hub-membership-types.tsx !' Join a paid tier

- ' Charged tier price + 95p platform fee (fee added on top). Hub receives full tier price (destination charge). DB: hubmembers row, memberno, paiduntil. Stripe: Connect !' hub account = tier price; platform keeps 95p.

Pass

Fail

Notes:

E2 — Donate (no Gift Aid) · Customer · hub-donate.tsx to an active campaign, £20

- ' Hub receives £20 (destination charge); platform takes ~Stripe cost only (no margin). DB: donation recorded.

Pass

Fail

Notes:

E3 — Donate with "cover the fees" · Customer · tick cover-fees, £20

- ' Customer pays £20 + fee estimate; hub nets the full £20.

Pass

Fail

Notes:

E4 — Donate with Gift Aid (charity hub) · Customer · full name + address + postcode, hub ischarity + charity number on file

- ' Gift Aid declaration stored; confirm returns giftaid: true. DB: Gift Aid fields recorded against the donation.

Pass

Fail

Notes:

E5 — Membership/donation on a hub NOT payout-ready

- ' Blocked with "has not finished setting up payouts yet"; no charge.

Pass

Fail

Notes:

F. Gifts & unit items (!' business payout)

F1 — Buy a unit item for self · Customer · local-buy-unit.tsx

- ' Charged price; commission (default 4%) to platform; rest destination-charged to business. DB: bookunitpurchases row; stock decremented. usesremaining/expiresat correct.

Pass

Fail

Notes:

F2 — Out-of-stock unit · buy a finite item with 0 stock

- ' "stockexhausted" error; no charge.

Pass

Fail

Notes:

F3 — Send a gift · Customer · local-gift.tsx, recipient email

- ' Charged; shareable code generated; recipient gets claim email. Re-running confirm doesn't double-charge or re-email (idempotent).

Pass

Fail

Notes:

G. Shift boost (platform-only revenue, no payout)

G1 — Boost a shift · Employer · shifts.tsx/my-posted-shifts !' Boost (£2.99)

- ' Charged £2.99 to platform (no destination charge); boosteduntil = now + 24h; matching workers notified.

Pass

Fail

Notes:

H. Business subscriptions & billing (recurring, platform revenue)

H1 — Subscribe Pro · Business owner · web /business/[id]/manage/billing !' Upgrade to Pro (£19.99)

- ' Stripe Checkout (subscription) opens !' pay !' returns. Webhook customer.subscription.created sets subscriptiontier=pro, subscriptionuntil, stripesubscriptionid.

Pass

Fail

Notes:

H2 — Change tier Pro !' Premium · Business owner · billing page

- ' Tier updates mid-cycle; DB reflects premium.

Pass

Fail

Notes:

H3 — Renewal · Stripe Dashboard !' advance the test clock / trigger invoice.paymentsucceeded

- ' subscriptionuntil extended.

Pass

Fail

Notes:

H4 — Cancel via billing portal · local-billing-portal !' Customer Portal !' cancel

- ' Webhook customer.subscription.deleted resets tier to free.

Pass

Fail

Notes:

H5 — Alert add-on (£10/mo) · Business owner (alert access approved) · subscribe

- ' Saved card !' activates server-side (activated:true); no card !' PaymentSheet.
businessalertaccess.status=active + stripesubscriptionid.

Pass

Fail

Notes:

I. PAYOUTS & Connect onboarding — test these hardest

Goal: prove money actually lands in the recipient's connected account and your platform fee is retained. After §C/D/E/F, open Stripe !' Connect !' the connected account !' Payments/Balance and confirm the figures.

I1 — Driver Connect onboarding · Driver · onboarding flow (create-connect-account)

- ' Express onboarding link opens; complete Stripe's test KYC. Webhook account.updated sets stripepayoutsenabled / chargesenabled = true. Driver shows "ready for payouts".

Pass

Fail

Notes:

I2 — Business Connect onboarding · Business owner · web payouts setup (local-business-onboard)

- ' Express onboarding completes; localbusinesses.payoutenabled=true.

Pass

Fail

Notes:

I3 — Hub Connect onboarding · Hub admin · web /hubs/[id]/manage/payouts (hub-onboard)

- ' Express onboarding completes; hubs.payoutenabled=true.

Pass

Fail

Notes:

I4 — Business payout received · after C3 (wallet pay) and F1 (unit sale)

- ' Business connected account balance = sum of transfers, each = amount " fee. Platform balance shows the application fees / retained commission.

Pass

Fail

Notes:

I5 — Hub payout received · after E1/E2 (membership/donation)

- ' Hub account = membership face value + donation; platform kept only the 95p / Stripe cost.

Pass

Fail

Notes:

I6 — Driver payout (Fetch end-to-end) · see §J below; confirm transfer lands on driver account.

Pass

Fail

Notes:

J. Fetch delivery (auth !' capture !' driver payout)

J1 — Authorise on accept · Customer card on file; Driver accepts a delivery

- ' PaymentIntent created with manual capture (authorised, not yet charged); transferdata.destination = driver's account; platform fee = £1.50 flat (default). Nothing captured yet.

Pass

Fail

Notes:

J2 — Capture on delivered (+ waiting fee) · Driver marks collected !' delivered (add a waiting fee if applicable)

- ' Full amount (base + waiting) captured; webhook paymentintent.succeeded sets deliveryrequests.paymentstatus=captured. Driver account receives amount " £1.50; platform keeps £1.50.

Pass

Fail

Notes:

J3 — Payment fails · use a card that fails capture

- ' paymentstatus=failed and the customer gets a push ("Delivery payment failed — please update your payment method"). (Now implemented via the webhook.)

Pass

Fail

Notes:

K. Negative / edge cases (do not skip before live)

K1. Every paid flow with a declined card !' no DB side-effects, clean error.

K2. Every paid flow with 3-D Secure card 4000 0025 0000 3155 !' auth challenge completes then succeeds.

K3. Double-tap / retry a confirm !' no double charge, no duplicate rows (idempotency).

K4. Payout-not-ready gates fire for business AND hub (C5/E5).

K5. Webhook signature: send an unsigned/garbage event !' rejected (fail-closed).

K6. Refunds (now in-app): Admin !' Payments !' a captured row !' Refund. ' Confirms via Stripe; for destination charges (tickets/donations/wallet pay-ins) it reverses the transfer + application fee so the money comes back from the recipient, not your platform balance. DB row flips to refunded; customer gets a "Refund issued" push. Also test a Dashboard refund !' the charge.refunded webhook flips the row too. (& p wallet-topup refunds don't auto-deduct wallet balance — handle those by adjusting the wallet, see notes.)

K7. Offer / reward / stamp redemption (non-payment) still works and can't be double-redeemed.

L. Webhook & data integrity sweep

L1. Stripe !' Webhooks !' no failing deliveries after a full test pass.

L2. Spot-check that each successful payment has a matching DB row (orders/tickets/transactions/memberships) — no "charged but not recorded".

L3. transfer.created events arrive (currently logged only — fine, just confirm no errors).

M. Go-live switch (only after A–L pass)

M1. Swap Supabase secrets to live keys: STRIPESECRETKEY (sklive...), STRIPEWEBHOOKSECRET, STRIPECONNECTWEBHOOKSECRET; web NEXTPUBLICSTRIPEPUBLISHABLEKEY (pklive...).

M2. Create a live webhook endpoint (same URL/events) and use its live signing secret.

M3. Create live recurring Price IDs and update adminconfig/env (...LOCALPRO/PREMIUM/ALERTADDON).

M4. Re-onboard real drivers/businesses/hubs through live Connect (test-mode accounts don't carry over).

M5. Confirm Stripe account is fully activated for live charges + Connect transfers (business details, bank, identity).

M6. One real low-value live transaction per flow (a £1 top-up, a cheap ticket, a £1 donation) + confirm a real payout lands, then refund/settle.

M7. Decide go-live stance on the known gaps: no refund UI (K6), no customer notice on delivery payment failure (J3), Gift-Aid postcode not validated.

Status of the earlier gaps (now addressed in code)

1. In-app refunds — ' added: Admin !' Payments !' Refund (refund-payment function), with Connect transfer/fee reversal + charge.refunded webhook. Remaining manual case: refunding a wallet top-up doesn't auto-deduct the wallet balance.
2. Delivery payment-failure — ' customer now gets a push (J3).
3. Subscription period-end — ' parsing hardened (item-level !' any item !' top-level, finite-checked).
4. Gift Aid postcode — ' validated + normalised on the app form and server-side (rejects invalid so Gift Aid isn't silently lost).
5. Missing Price IDs — error was already clear server-side and the client now surfaces it; just set the IDs (A2).

Deploy before testing these: supabase functions deploy refund-payment stripe-webhook confirm-hub-donation, and add charge.refunded to your Stripe webhook's event list.

Fee defaults (overridable in adminconfig): Fetch £1.50 flat · Wallet 2% + 25p · Products/units/gifts 4% · Hub membership +95p · Donations ~Stripe cost only · Subscriptions/boosts/alerts = platform keeps 100%.